# Chapter - 3

# CENTRAL PROCESSING UNIT

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# CPU ORGANIZATION AND STRUCTURE

## *Components of the CPU*

**Control Unit (CU)**

- "nerve centre" of CPU
- send control signals to other components
- sense their states to coordinate their operations

**Arithmetic and Logic Unit (ALU)**

- perform arithmetical, comparison and logical operation
- work closely with the accumulation

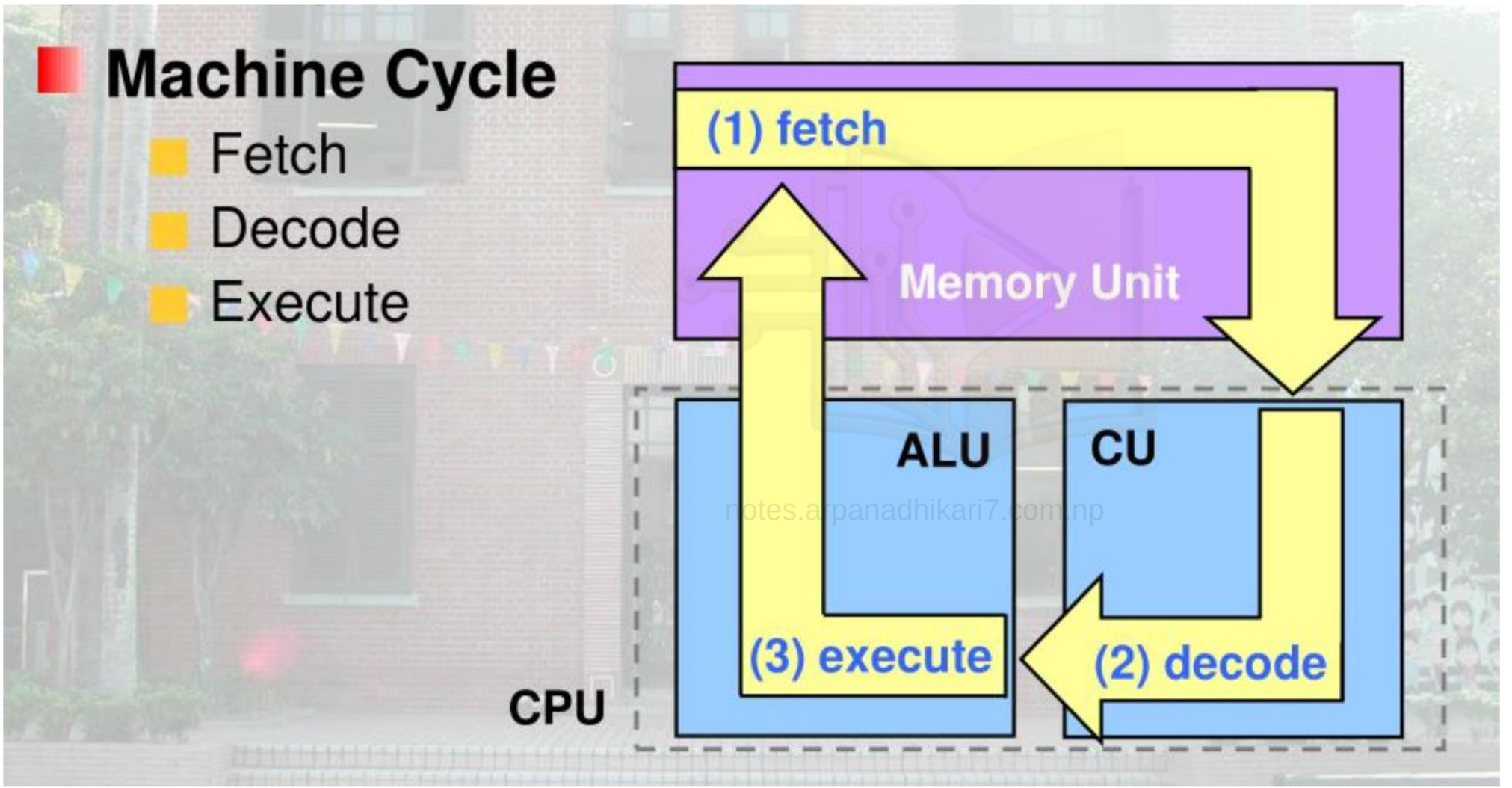
notes.arpanadhikari7.com.np

## Register

- high-speed storage locations
- temporarily hold data and instructions during processing

Available on notes.arpanadhikari7.com.np

# MAJOR COMPONENTS OF CPU




notes.arpanadhikari7.com.np

# Processor Organization

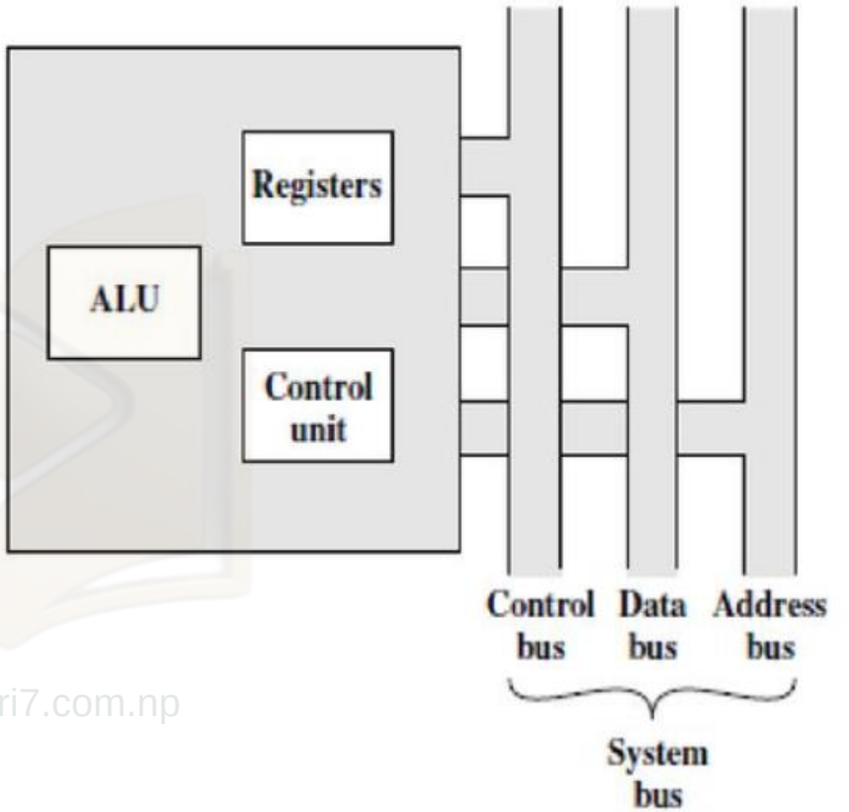
- Requirements of CPU:
  1. Fetch instructions (from m/m,reg,cache)
  2. Interpret instructions (decoding)
  3. Fetch data
  4. Process data/Execute (arithmetic/logical operations)
  5. Write data

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

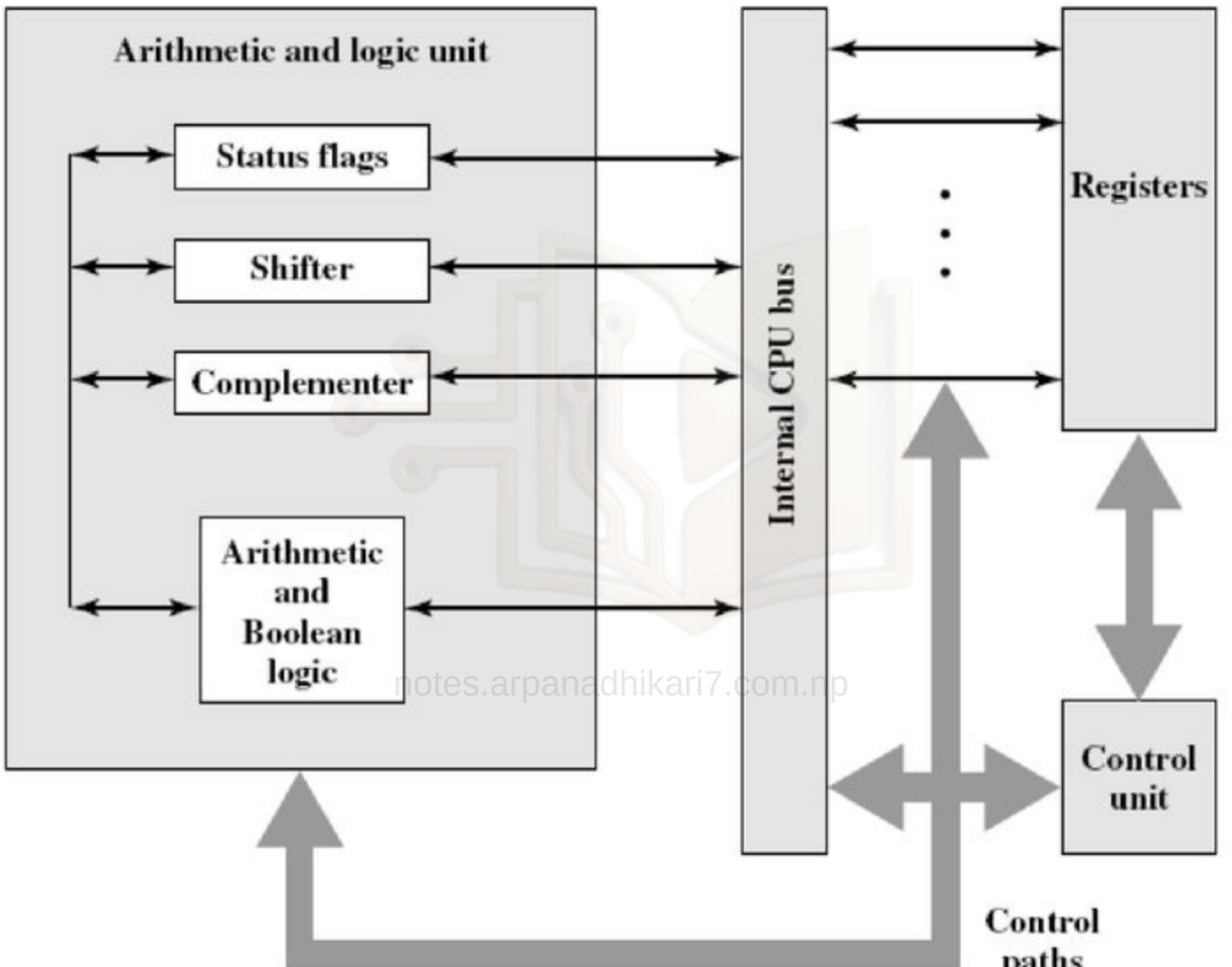
# Processor Organization

- It connects rest of system via system bus
- The major components of processor are
  - ALU does processing
  - CU controls movement of data and ALU
  - Registers (Internal Memory consisting of a set of storage locations)




Available on notes.arpanadhikari7.com.np

# Internal Structure of CPU




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Internal Structure of CPU

- It includes data transfer (between registers and ALU) and logic control paths.
- It also includes an internal CPU bus to transfer data between ALU and  Registers

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Register Organization

- The registers in the processor performs two roles:

1. **User Visible Registers :**
   - Minimize main memory references by optimizing use of registers
2. **Control and Status Registers:**
   - Used by control unit to control the operation of processor

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# User Visible Registers

- Referenced by means of machine language
- They are categorized as :
  - General Purpose
  - Data
  - Address
  - Condition Codes

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# General Purpose Registers

- Contain operands
- Can be used for addressing functions

# Data Registers

- Used only to hold data

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Address Registers

- may be general purpose or
- devoted to an addressing mode
  - Example:
    - **Segment Pointers:** holds base address of segment
    - **Index Registers:** for indexed addressing
    - **Stack Pointer:** points to the top of stack.

## Condition codes (Flags)

Bits set by the processor hardware as the result of operations

E.g. Carry,Overflow,Sign

They may be tested as part of conditional branch operation

Condition code bits are collected into one or more registers but the programmer cannot alter them.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Control and Status Registers

- **Control Registers :** Four registers essential for instruction execution are
  - Program Counter
  - Instruction Register
  - Memory Address Register
  - Memory Buffer Register

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Status Register

- A register or set of registers that contain status information is called Program Status Word (PSW)
- PSW contains condition codes and other status information
- PSW contains :
  - **Sign:** Sign bit of last operation
  - **Zero:** Set when result is 0.
  - **Carry:** Set when a carry/borrow is generated /taken.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Status Register

- PSW contains :
  - **Overflow:** When arithmetic overflow occurs
  - **Interrupt Enable/Disable:** Used to enable / disable interrupts
  - **Supervisor:** Indicates whether in supervisor/user mode.
- **Other registers are :** interrupt vector register, system stack pointer

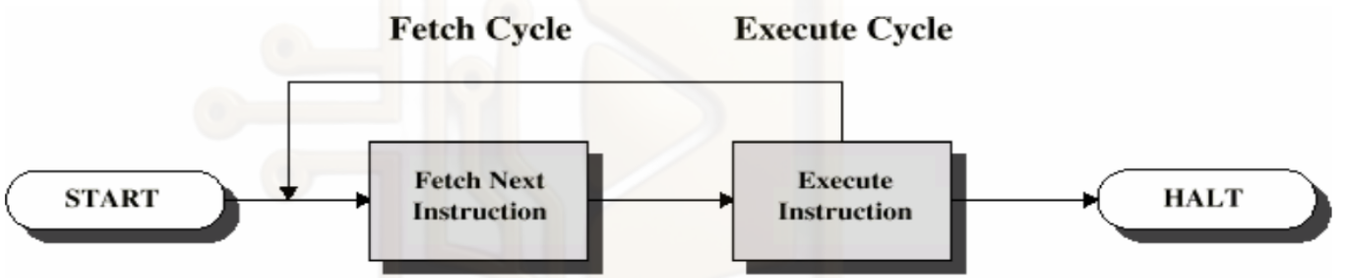
Available on notes.arpanadhikari7.com.np

# INSTRUCTION CYCLE

- Instruction cycle is not the same thing as clock cycle

Two Steps:

- FETCH: Read the next instruction from memory into processor.
- EXECUTE: Interpret the op-code and perform the indicated operation.

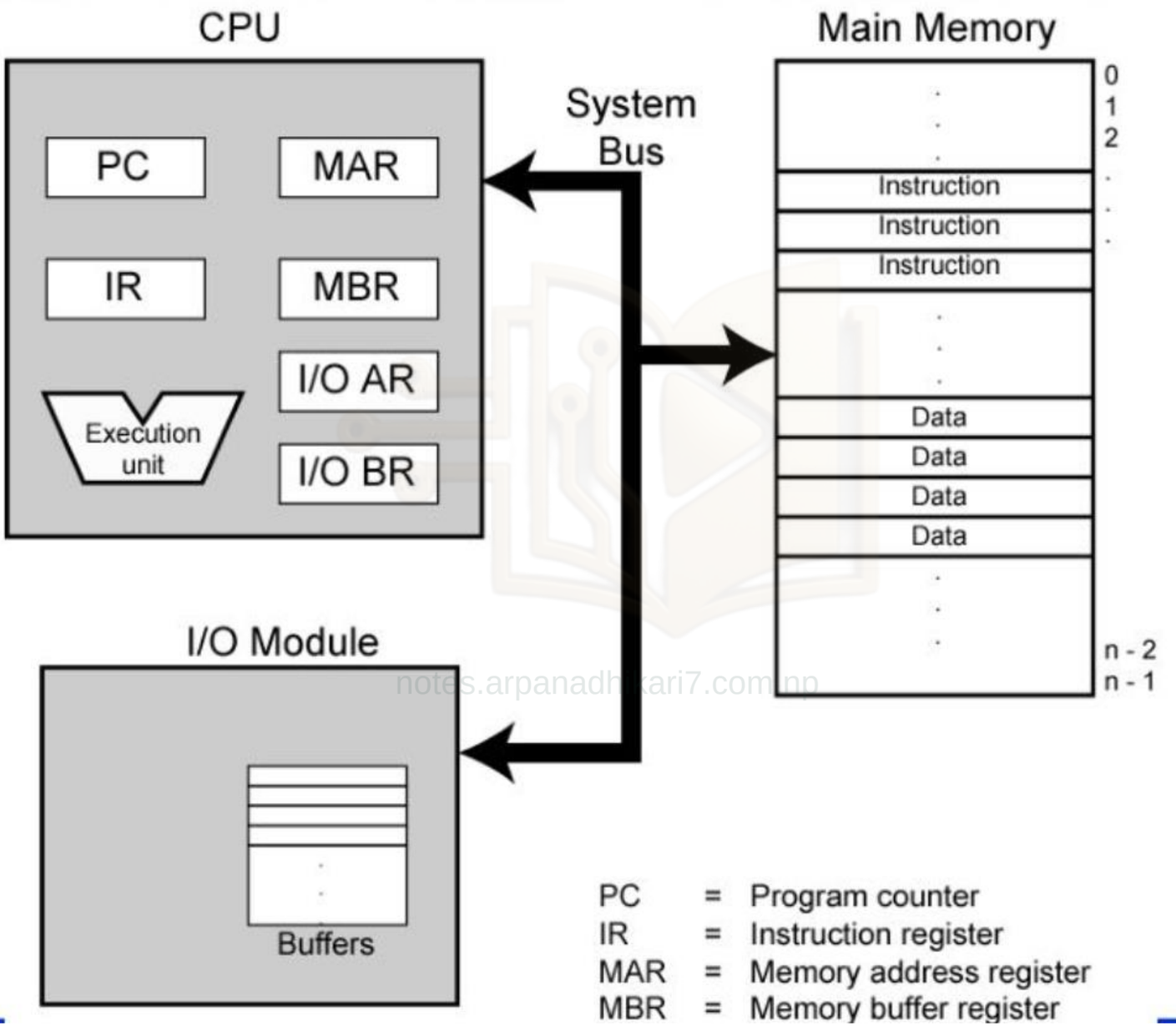



Program execution halts only if the machine is turned off, some sorts of unrecoverable error occurs, or a program instruction that halts the computer is encountered.

Figure: Basic Instruction cycle

Available on notes.arpanadhikari7.com.np

# Instruction Cycle




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Press Esc to exit full screen

# Fetch Cycle

- Program Counter (PC) holds address of next instruction to fetch
- Processor fetches instruction from memory location pointed to by PC
- Increment PC
  - Unless told otherwise
- Instruction loaded into Instruction Register (IR)
- Processor interprets instruction and performs required actions

Available on notes.arpanadhikari7.com.np

Press Esc to exit full screen

# Execute Cycle

- **Processor-memory**
  - data transfer between CPU and main memory
- **Processor I/O**
  - Data transfer between CPU and I/O module
- **Data processing**
  - Some arithmetic or logical operation on data
- **Control**
  - Alteration of sequence of operations
  - e.g. jump
- **Combination of above**

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

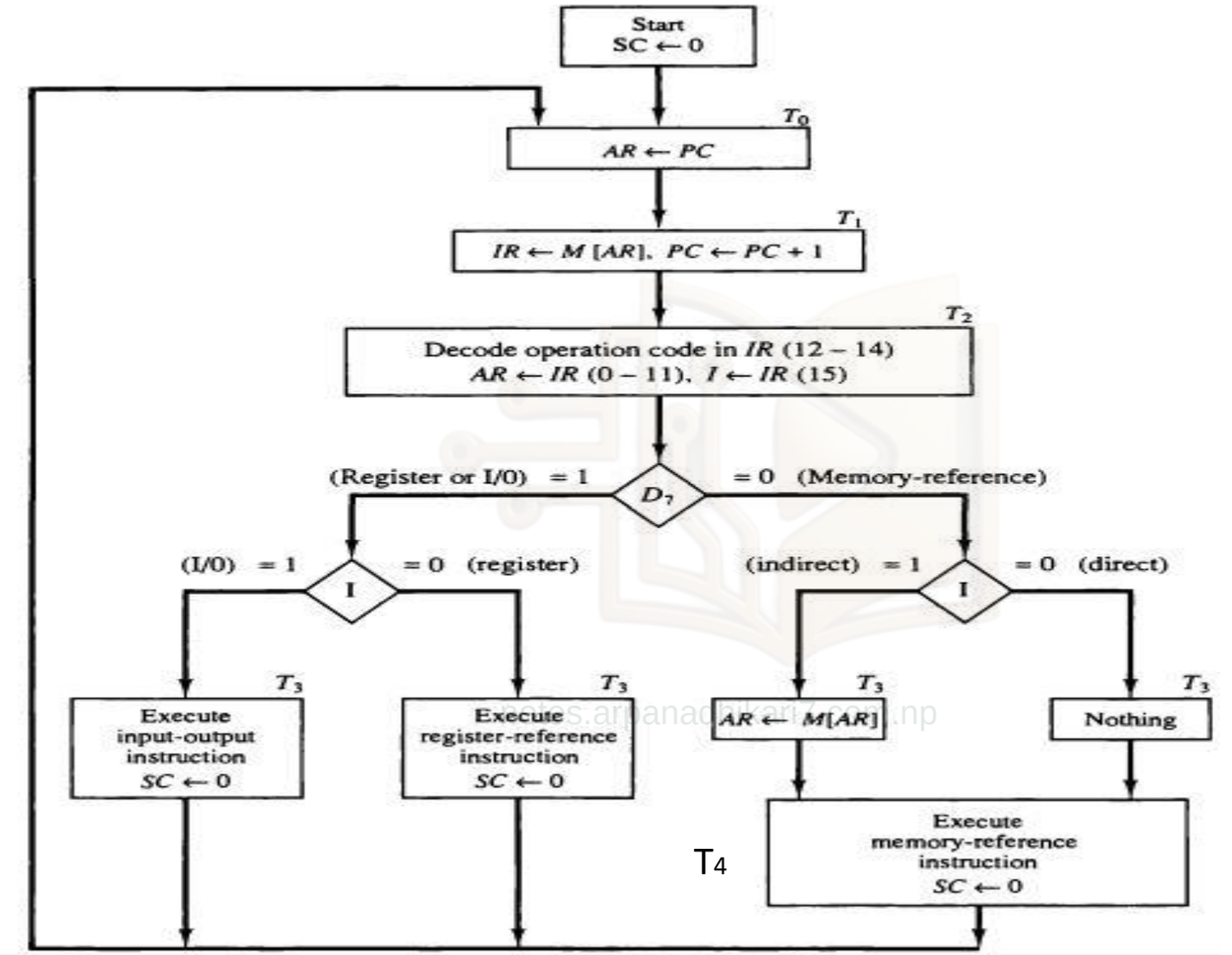
# Instruction execution

- Determine the address of the next instruction
- Fetch that instruction from memory
- Decode the instruction to determine what is to be performed
- Calculate the addresses of needed operands and fetch the operands
- Perform the operation on the operands
- Store the results
- Check for and service pending interrupts

notes.arpanadhikari7.com.np

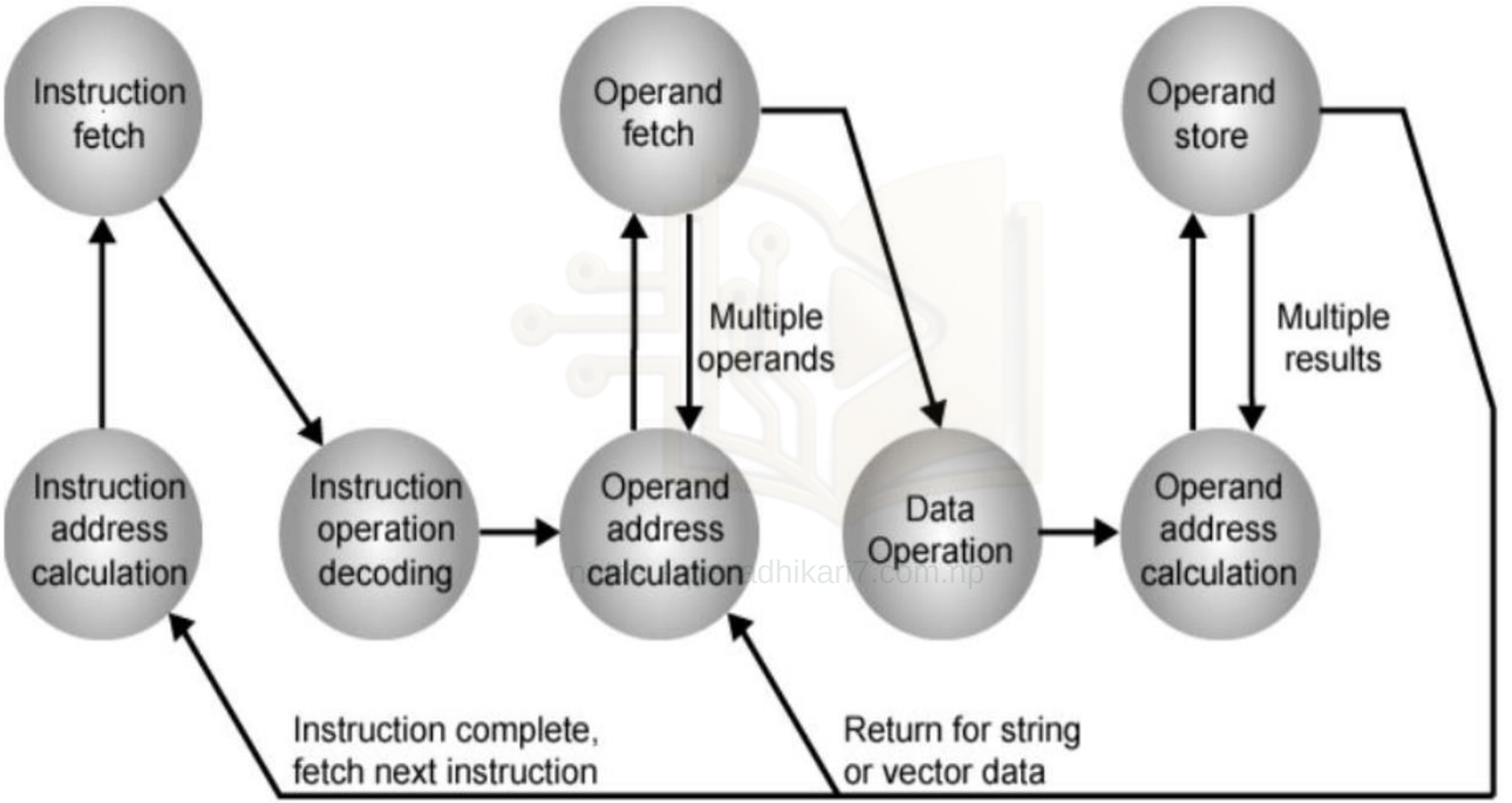
Available on notes.arpanadhikari7.com.np

# Instruction Cycle




Available on notes.arpanadhikari7.com.np

# Instruction Cycle State Diagram




Available on notes.arpanadhikari7.com.np

# Instruction Cycle State Diagram

- Instruction address calculation (iac): Determine the address of the next instruction to be executed. Usually, this involves adding a fixed number to the address of the previous instruction. For example, if each instruction is 16 bits long and memory is organized into 16-bit words, then add 1 to the previous address. If, instead, memory is organized as individually addressable 8-bit bytes, then add 2 to the previous address.
- Instruction fetch (if): Read instruction from its memory location into the processor.
- Instruction operation decoding (iod): Analyze instruction to determine type of operation to he performed and operand(s) to be used.
- Operand address calculation (oac): If the operation involves reference to an operand in memory or available via I/O. then determine the address of the operand.
- Operand fetch (of): Fetch the operand from memory or read it in from I/O
- Data operation (do): Perform the operation indicated in the instruction.
- Operand store (os): Write the result into memory or out to I/O

Available on notes.arpanadhikari7.com.np

# Instruction Cycle (Interrupt)

## Interrupts

- Mechanism by which other modules (e.g. I/O) may interrupt normal sequence of processing
- Program
  - e.g. overflow, division by zero
- Timer
  - Generated by internal processor timer
  - Used in pre-emptive multi-tasking
- I/O
  - from I/O controller
- Hardware failure
  - e.g. memory parity error

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

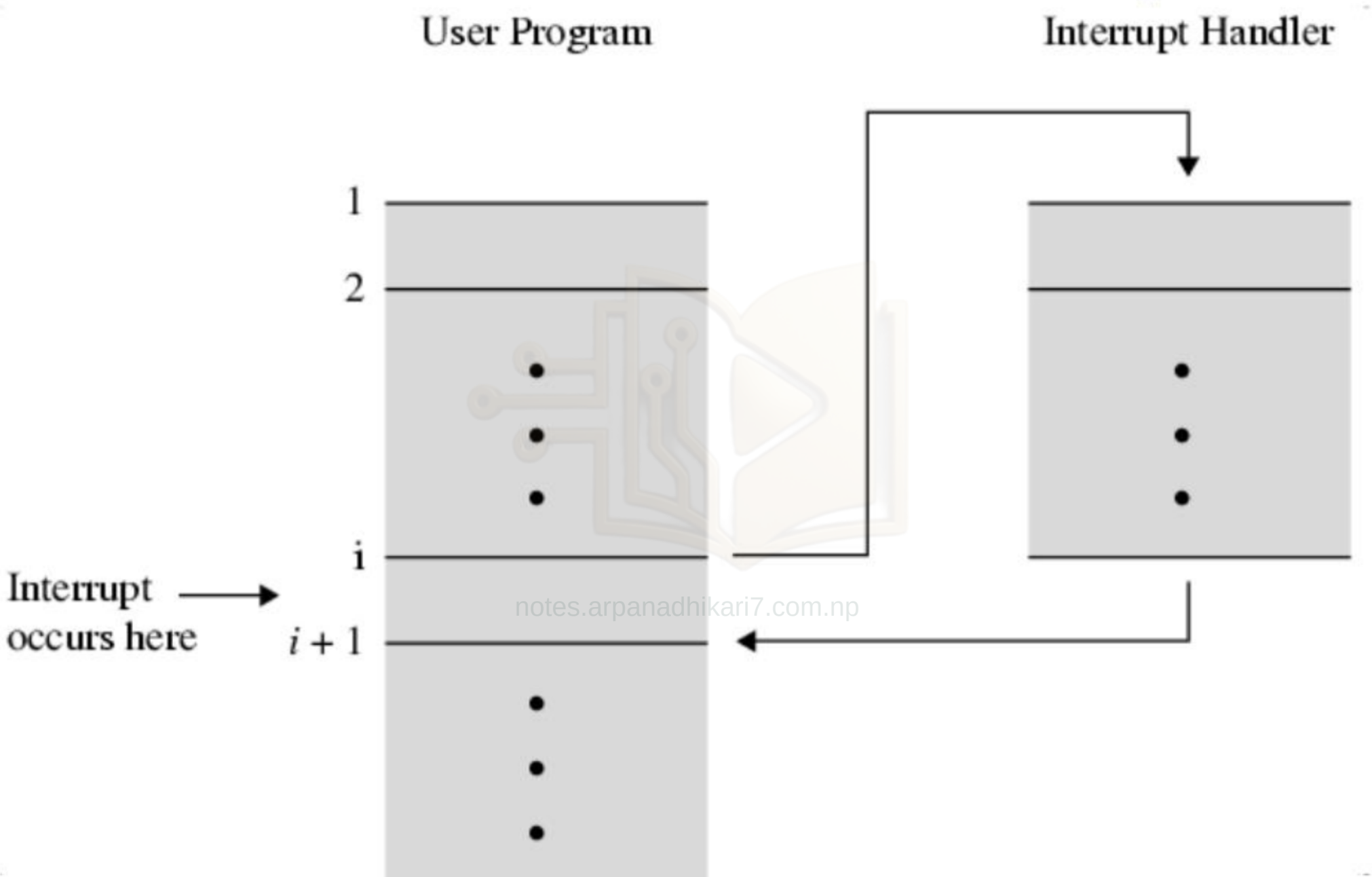
# Interrupt Cycle

- Added to instruction cycle
- Processor checks for interrupt
  - Indicated by an interrupt signal
- If no interrupt, fetch next instruction
- If interrupt pending:
  - Suspend execution of current program
  - Save context
  - Set PC to start address of interrupt handler routine
  - Process interrupt
  - Restore context and continue interrupted program

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

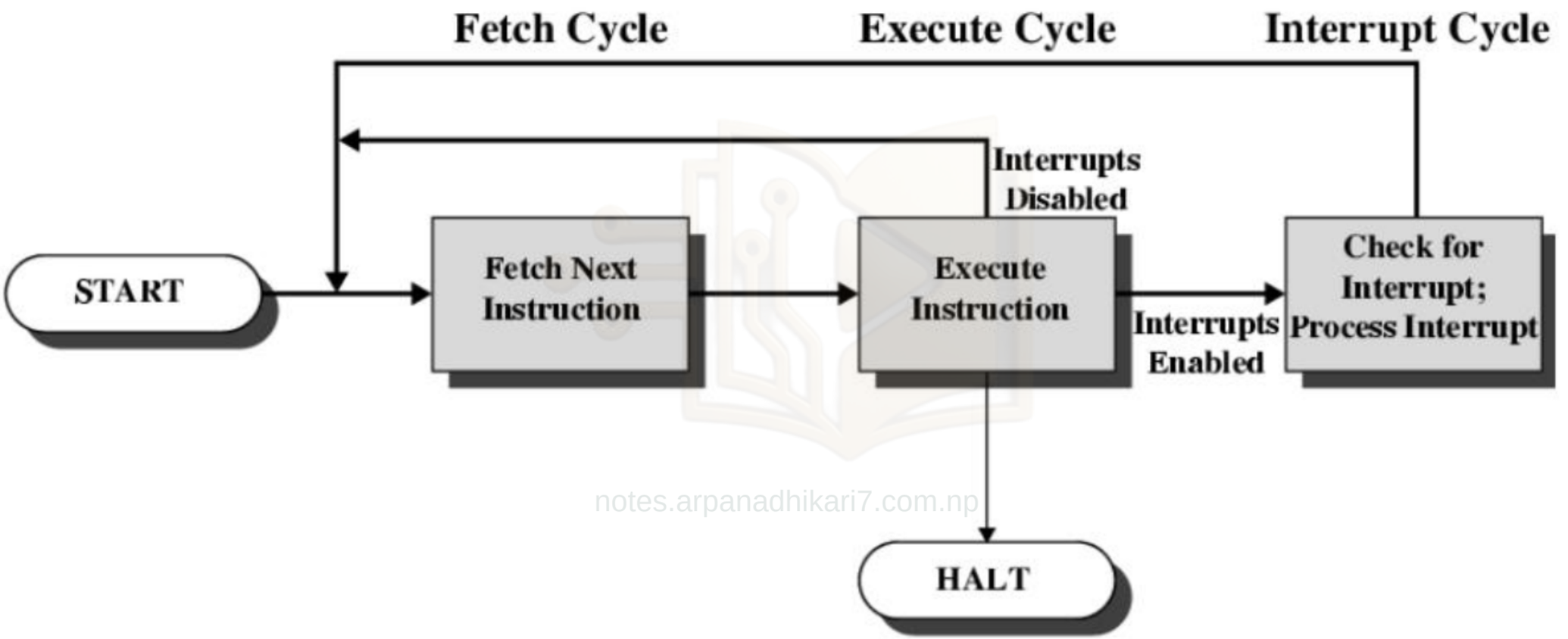
# Transfer of Control via Interrupts




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

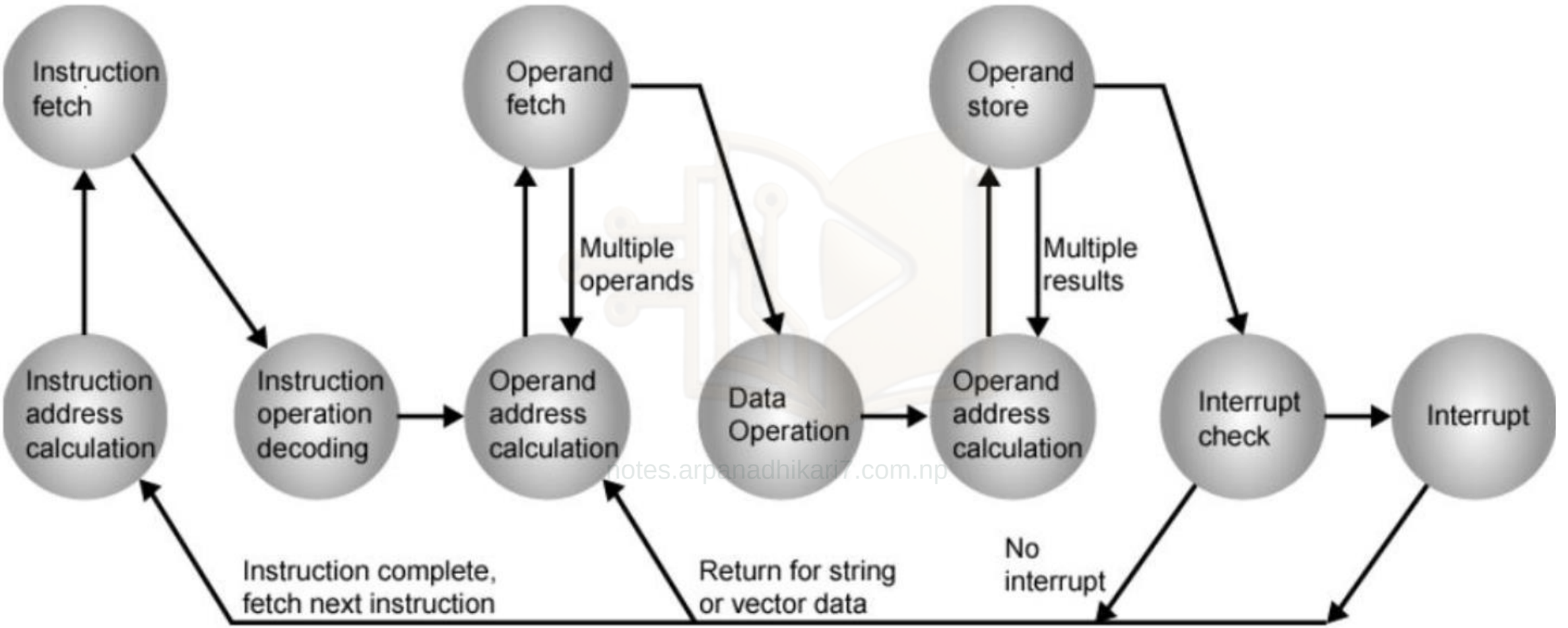
# Instruction Cycle with Interrupts




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Instruction Cycle (with Interrupts) - State Diagram




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **The Indirect Cycle**

- Another instruction sub-cycle
- May require just another fetch (based upon last fetch)
- Might also require arithmetic, like indexing

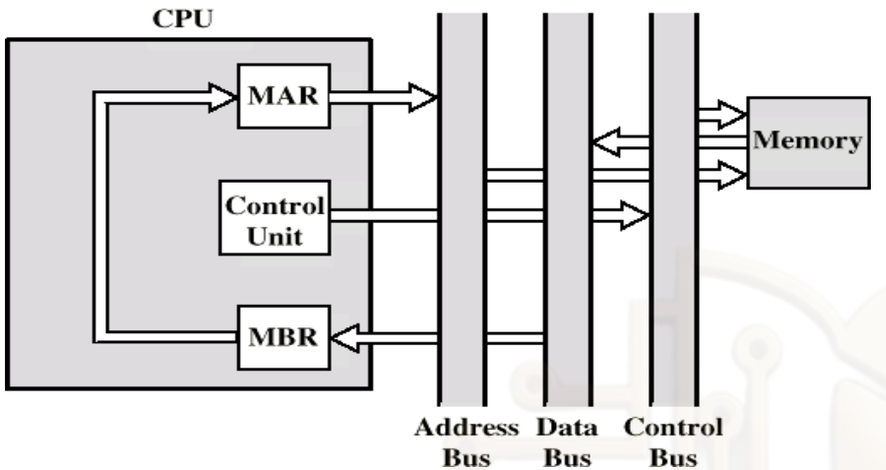



Fig: Data Flow, Indirect Cycle

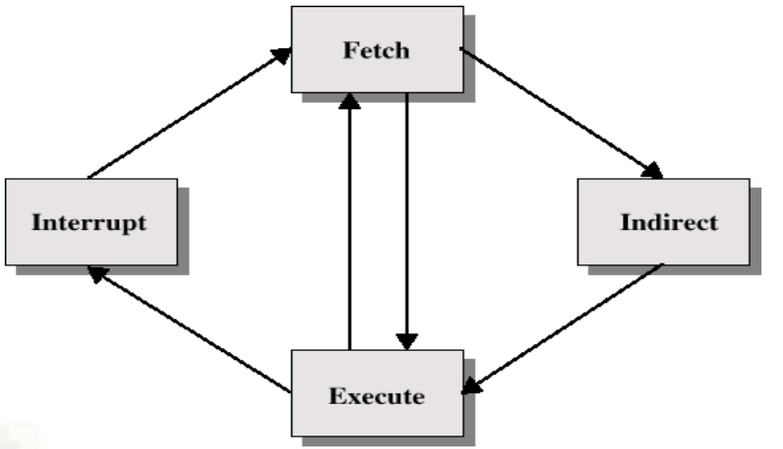



Fig: Instruction Cycle with Indirect

**Indirect cycle data flow**

- Decodes the instruction
- After fetch, control unit examines IR to see if indirect addressing is being used. If so:
- Rightmost n bits of MBR (the memory reference) are transferred to MAR
- Control unit requests a memory read, to get the desired operand address into the MBR
- t1: MAR ← (IR(Address))
- t2: MBR ← Memory
- t3: IR(Address) ← (MBR(Address))

Available on notes.arpanadhikari7.com.np

- **Interrupt cycle data flow**

- Current contents of PC must be saved (for resume after interrupt), so PC is transferred to MBR to be written to memory
- Save location's address (such as a stack ptr) is loaded into MAR from the control unit
- PC is loaded with address of interrupt routine (so next instruction cycle will begin by fetching appropriate instruction)
    - t1: MBR ← (PC)
    - t2: MAR ← save address
      PC ← Routine address
    - t3: Memory ← (MBR)

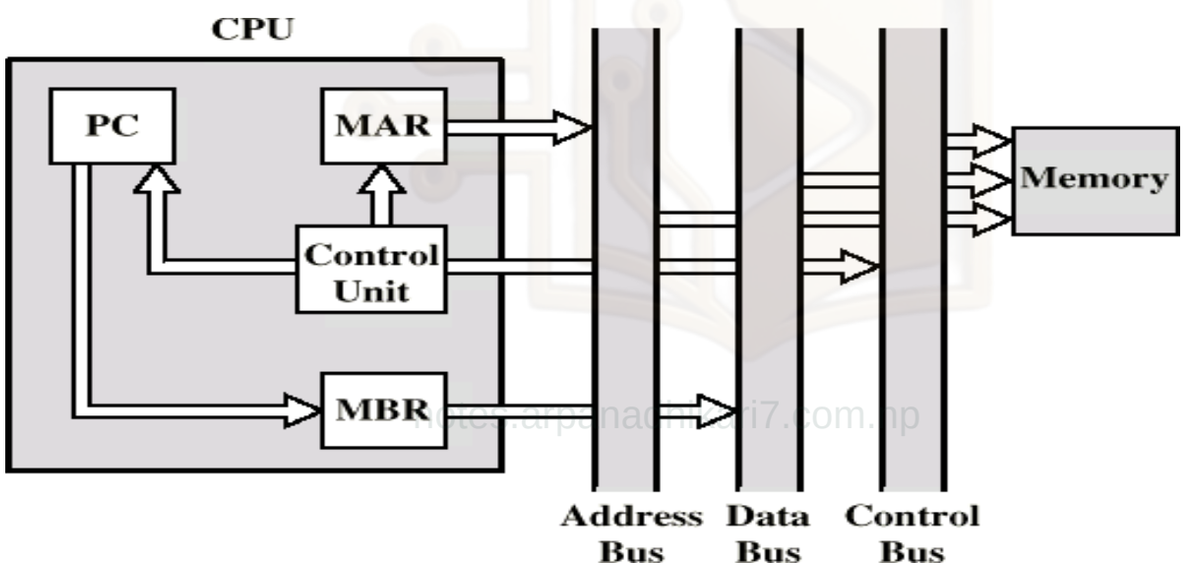



Fig: Data Flow, Interrupt Cycle

Available on notes.arpanadhikari7.com.np

# Arithmetic and Logical Unit

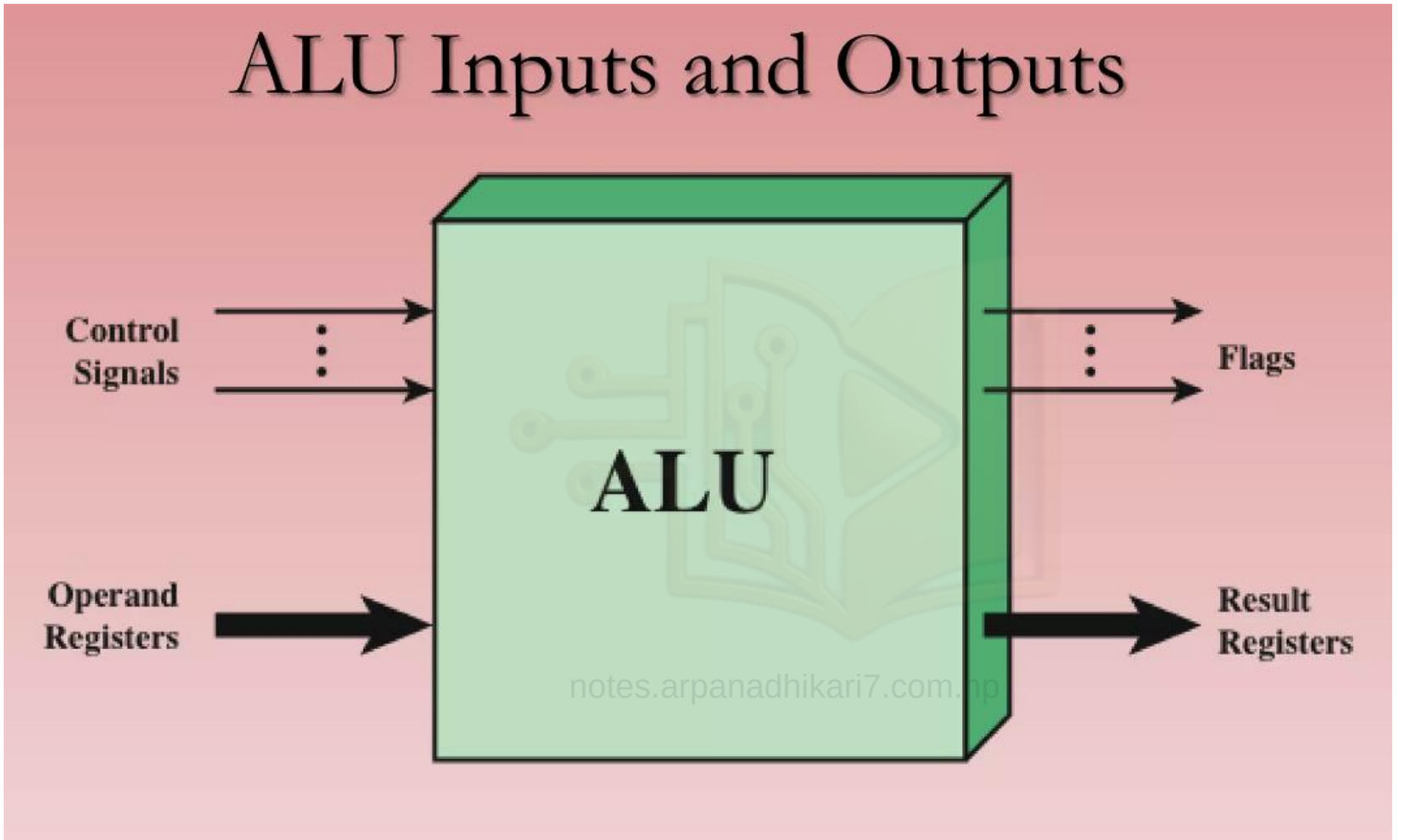



Figure: ALU inputs and outpts

Available on notes.arpanadhikari7.com.np

- ALU is the combinational circuit of that part of computer that actually performs arithmetic and logical operations on data.
- All of the other elements of computer system- control unit, registers, memory, I/O are their mainly to bring data into the ALU for it to process and then to take the result back out.
- An ALU & indeed all electronic components in computer are based on the use of simple digital logic device that can store binary digit and perform simple Boolean logic function.
- Data are presented to ALU in register and the result of operation is stored in register.
- The ALU may also set flags as the result of an operation and values are also stored in registers within the processor.
- The control unit provides signals that control the operation of ALU and the movement of data into an out of ALU.

The design of ALU has three stages.

1. **Design the arithmetic section**
2. **Design of logic section**
3. **Combine these two section to form ALU**

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1. **Design the arithmetic section**
   - The basic component of arithmetic circuit is a parallel adder which is constructed with a number of full adder circuits connected in cascade. By controlling the data inputs to the parallel adder, it is possible to obtain different types of arithmetic operations. Below figure shows the arithmetic circuit and its functional table.

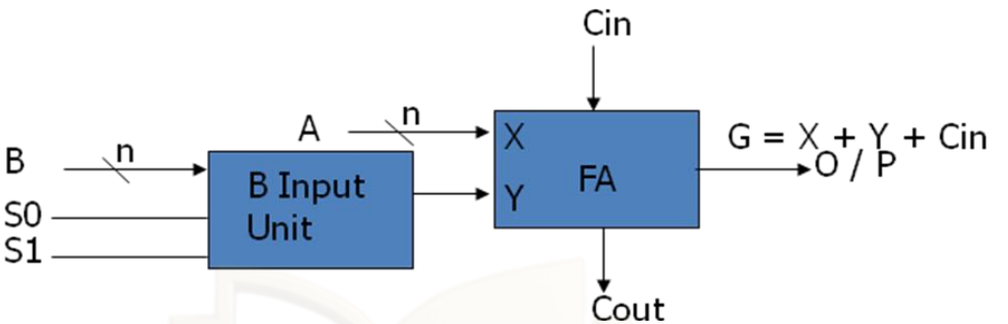



Fig: Block diagram of Arithmetic Unit

Functional table for arithmetic unit:

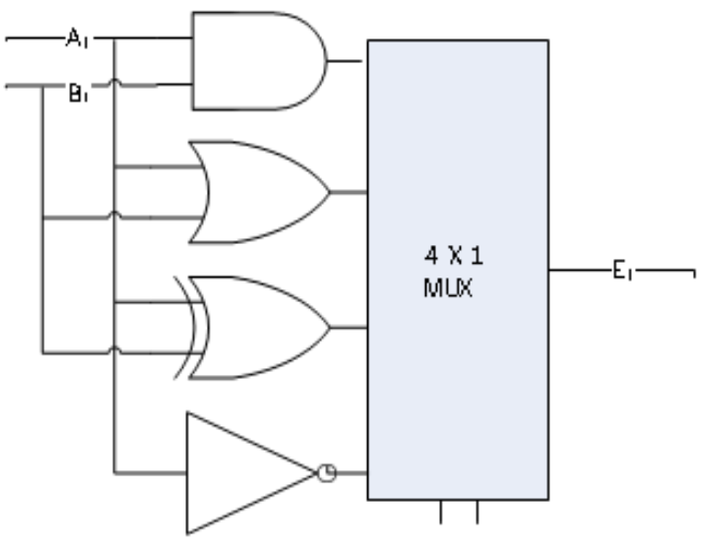
| Select | | Input | Output | | Microoperation | |
|---|---|---|---|---|---|---|
| $S_1$ | $S_0$ | Y | Cin = 0 | Cin = 1 | Cin = 0 | Cin = 1 |
| 0 | 0 | 0 | A | A+1 | Transfer A | Increment A |
| 0 | 1 | B | A+B | A+B+1 | Addition | Addition with carry |
| 1 | 0 | B' | A+B' | A+B'+1 | Subtraction with borrow | Subtraction |
| 1 | 1 | -1 | A-1 | A | Decrement A | Transfer A |

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

**Design the logical section**

- The basic components of logical circuit are AND, OR, XOR and NOT gate circuits connected accordingly. Below figure shows a circuit that generates four basic logic micro-operations. It consists of four gates and a multiplexer. Each of four logic operations is generated through a gate that performs the required logic. The two selection input S1 and S0 choose one of the data inputs of the multiplexer and directs its value to the output. Functional table lists the logic operations.




Functional table for logic unit:

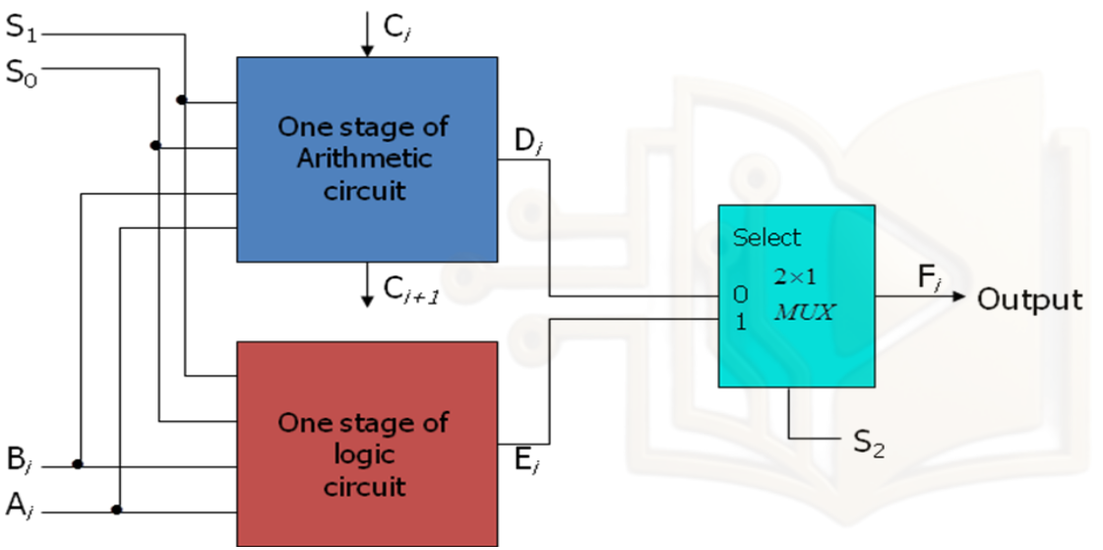
| S1 | S0 | output | Microoperation |
|---|---|---|---|
| 0 | 0 | Ai && Bi | AND |
| 0 | 1 | Ai \|\| Bi | OR |
| 1 | 0 | Ai XOR Bi | XOR |
| 1 | 1 | Ai' | NOT |

notes.arpanadhikari7.com.np

Fig: Block diagram of Logic Unit

Available on notes.arpanadhikari7.com.np

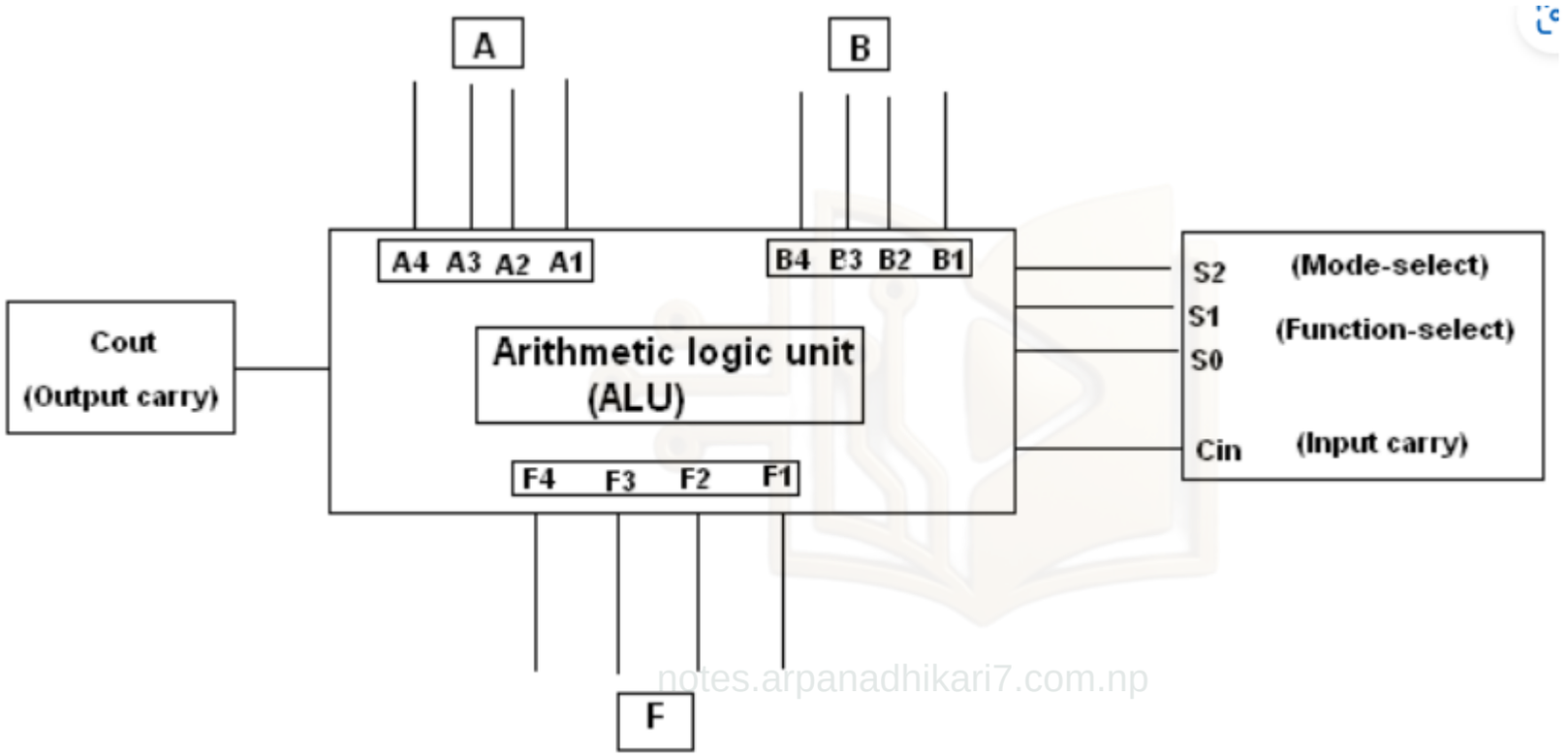
- ALU has a number of selection lines used to determine the operation to be performed. The selection lines are decoded with the ALU so that selection lines can specify distinct operations.
- The mode select $S_2$ differentiate between arithmetic and logical operations. The two functions select $S_1$ and $S_0$ specify the particular arithmetic and logic operationsto be performed. With three selection lines, it is possible to specify arithmetic operation with $S_2$ at 0 and logical operation with $S_2$ at 1




notes.arpanadhikari7.com.np

Fig: Block diagram of ALU

Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Figure: Block diagram of 4-bit ALU

Available on notes.arpanadhikari7.com.np

# Design of 1- bit ALU

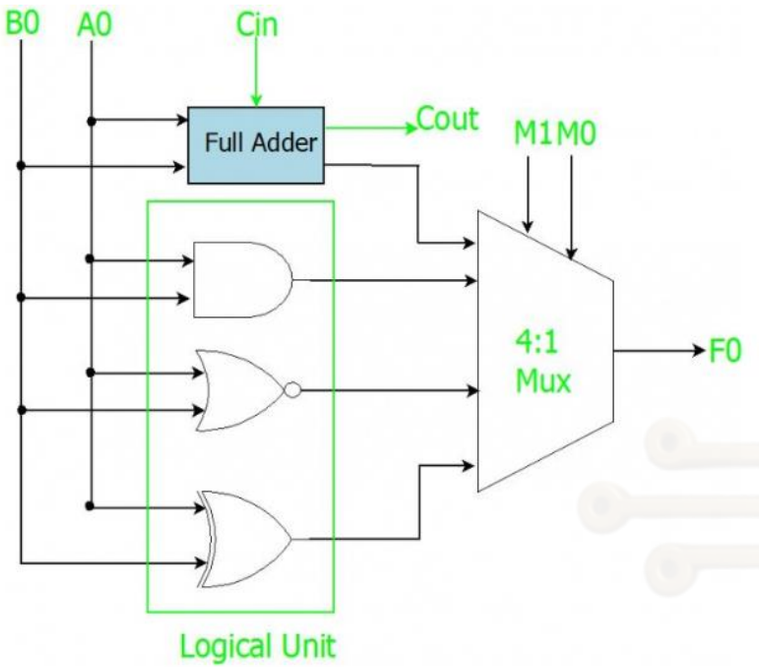
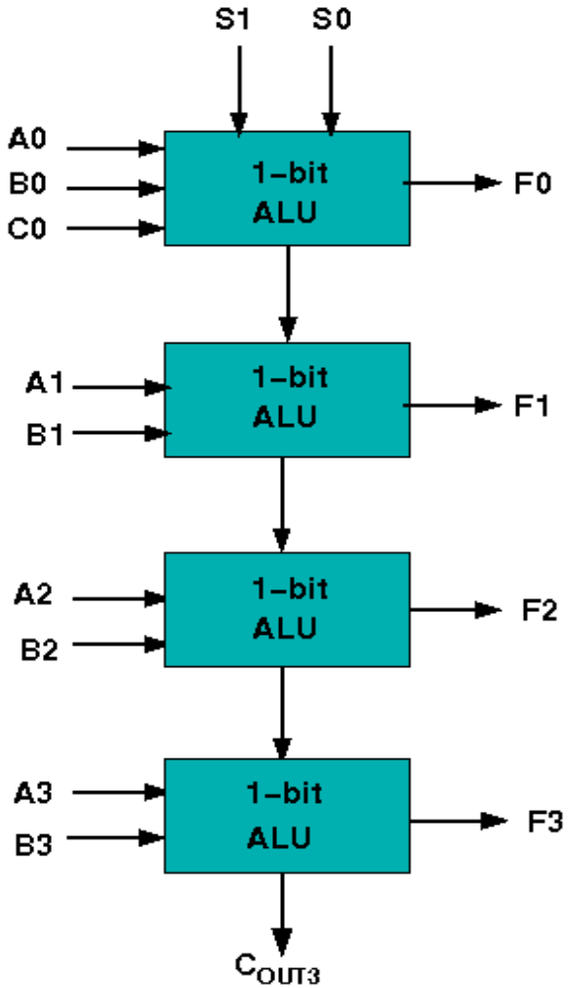



Figure: 1 bit ALU

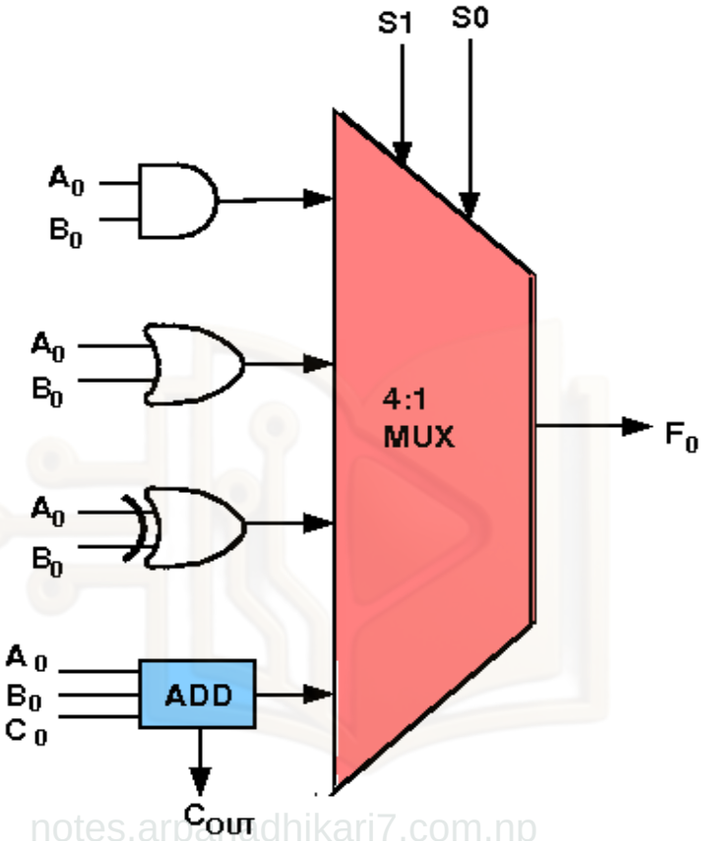
notes.arpanadhikari7.com.np

| Inputs | | Outputs |
|---|---|---|
| ***M1*** | ***M0*** | ***Operation*** |
| 0 | 0 | SUM |
| 1 | 0 | AND |
| 0 | 1 | OR |
| 1 | 1 | XOR |

# Design of ALU




4-bit ALU designed by using 4 one bit ALU




1-bit ALU

**Design Issues :**

The circuit functionality of a 1 bit ALU is shown here, depending upon the control signal $S_1$ and $S_0$ the circuit operates as follows:

for Control signal $\mathbf{S_1 = 0}$ , $S_0 = 0$, the output is **A And B**,

for Control signal $\mathbf{S_1 = 0}$ , $S_0 = 1$, the output is **A Or B**,

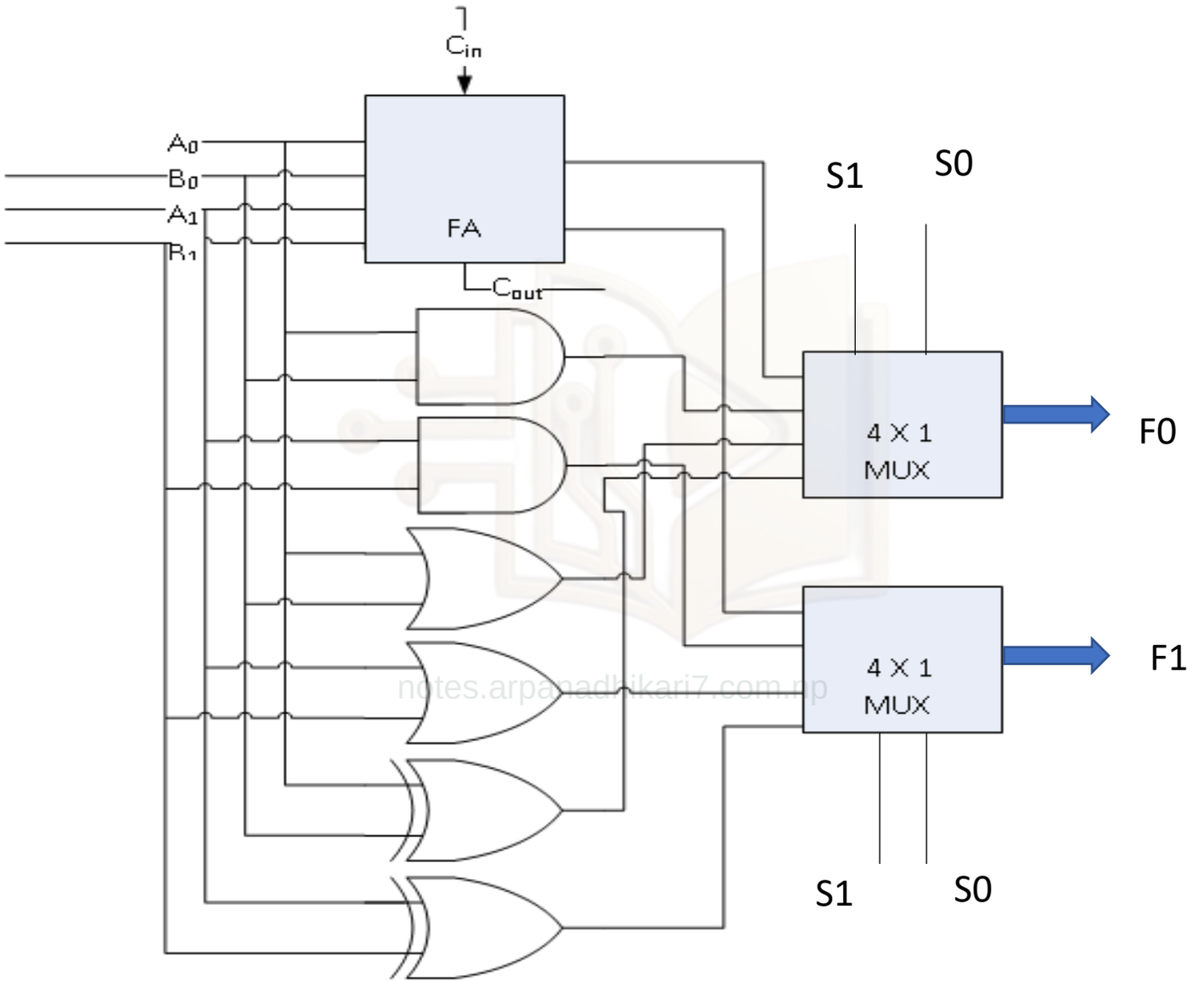
for Control signal $\mathbf{S_1 = 1}$ , $S_0 = 0$, the output is **A Xor B**,

for Control signal $\mathbf{S_1 = 1}$ , $S_0 = 1$, the output is **A Add B**.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

**Example:** Design a 2-bit ALU that can perform addition, AND, OR, & XOR.




notes.arpanadhikari7.com.np

## 9-4 DESIGN OF ARITHMETIC CIRCUIT

The basic component of the arithmetic section of an ALU is a parallel adder. A parallel adder is constructed with a number of full-adder circuits connected in cascade (see Section 5-2). By controlling the data inputs to the parallel adder, it is possible to obtain different types of arithmetic operations. Figure 9-6 demonstrates the arithmetic operations obtained when one set of inputs to a parallel adder is controlled externally. The number of bits in the parallel adder may be of any value. The input carry $C_{in}$ goes to the full-adder circuit in the least significant bit position. The output carry $C_{out}$ comes from the full-adder circuit in the most significant bit position.

The arithmetic addition is achieved when one set of inputs receives a binary number $A$, the other set of inputs receives a binary number $B$, and the input carry is maintained at 0. This is shown in Fig. 9-6(a). By making $C_{in} = 1$ as in Fig.



Available on notes.arpanadhikari7.com.np

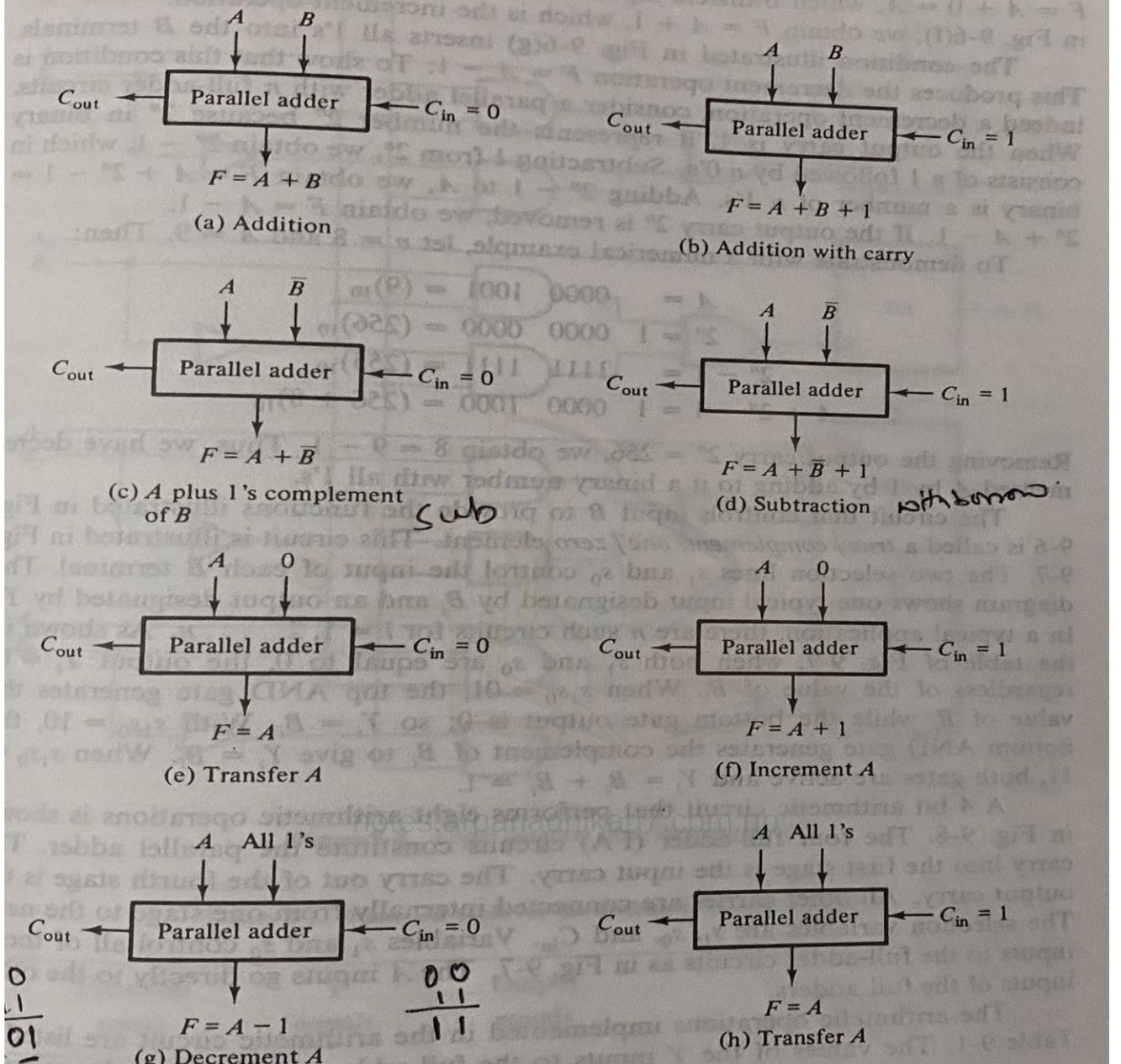



**Figure 9-6** Operations obtained by controlling one set of inputs to a parallel adder

Now consider the effect of

Available on notes.arpanadhikari7.com.np

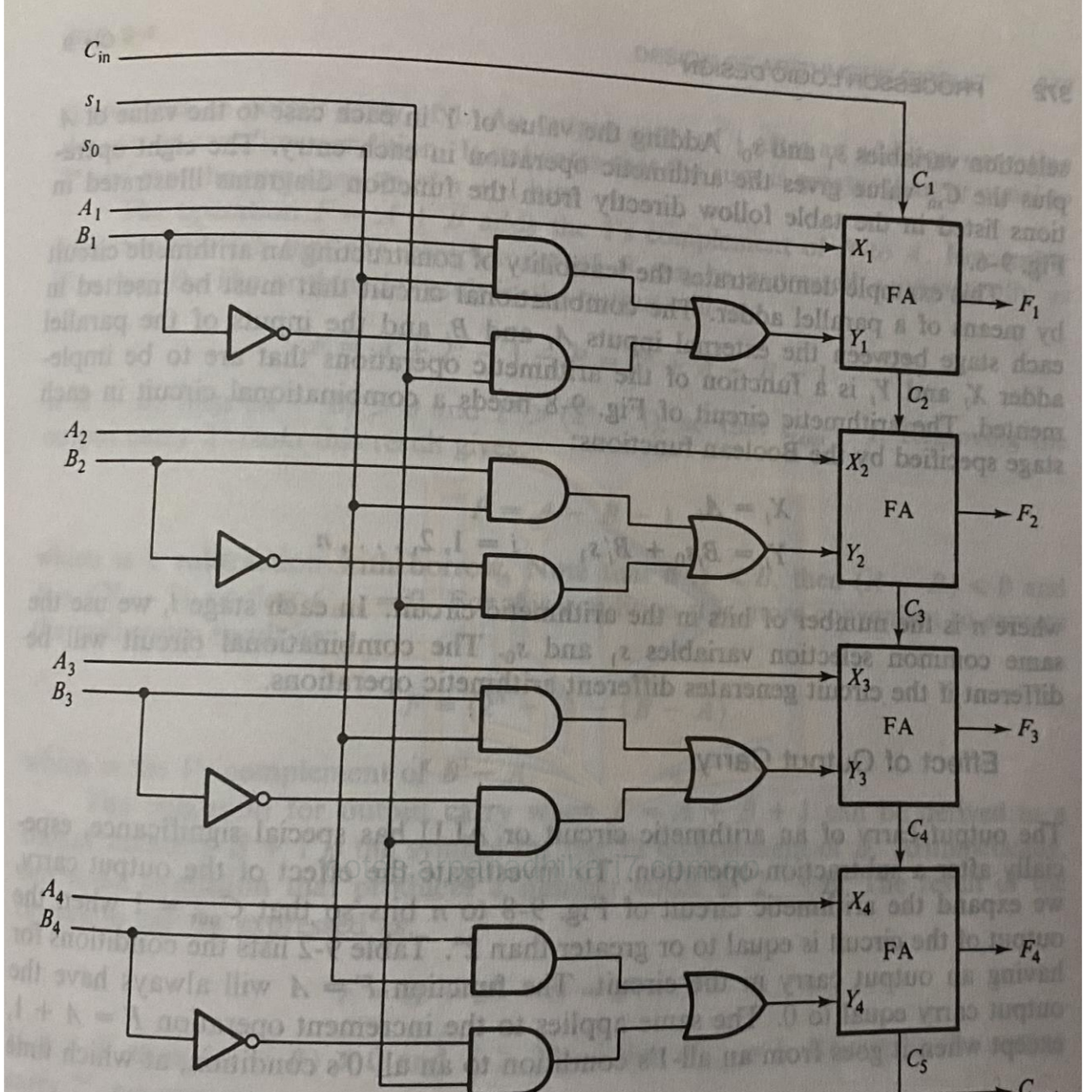



**Figure 9-8** Logic diagram of arithmetic circuit

Available on notes.arpanadhikari7.com.np

## 9-6 DESIGN OF ARITHMETIC LOGIC UNIT

In this section, we design an ALU with eight arithmetic operations and four logic operations. Three selection variables $s_2$, $s_1$, and $s_0$ select eight different operations, and the input carry $C_{in}$ is used to select four additional arithmetic operations. With $s_2 = 0$, selection variables $s_1$ and $s_0$ together with $C_{in}$ will select the eight arithmetic operations listed in Table 9-1. With $s_2 = 1$, variables $s_1$ and $s_0$ will select the four logic operations OR, XOR, AND, and NOT.

The design of an ALU is a combinational-logic problem. Because the unit has a regular pattern, it can be broken into identical stages connected in cascade through the carries. We can design one stage of the ALU and then duplicate it for the number of stages required. There are six inputs to each stage: $A_i$, $B_i$, $C_i$, $s_2$, $s_1$, and $s_0$. There are two outputs in each stage: output $F_i$ and the carry out $C_{i+1}$. One can formulate a truth table with 64 entries and simplify the two output functions.

Available on notes.arpanadhikari7.com.np

Here we choose to employ an alternate procedure that uses the availability of a parallel adder.

The steps involved in the design of an ALU are as follows:

1. Design the arithmetic section independent of the logic section.
2. Determine the logic operations obtained from the arithmetic circuit in step 1, assuming that the input carries to all stages are 0.
3. Modify the arithmetic circuit to obtain the required logic operations.

The third step in the design is not a straightforward procedure and requires a certain amount of ingenuity on the part of the designer. There is no guarantee that a solution can be found or that the solution uses the minimum number of gates. The example presented here demonstrates the type of logical thinking sometimes required in the design of digital systems.

It must be realized that various ALUs are available in IC packages. In a practical situation, all that one must do is search for a suitable ALU or processor unit among the ICs that are available commercially. Yet, the internal logic of the IC selected must have been designed by a person familiar with logic design techniques.

The solution to the first design step is shown in Fig. 9-8. The solution to the second design step is presented in Table 9-3. The solution of the third step is carried out below.

From Table 9-3, we see that when $s_2 = 1$, the input carry $C_i$ in each stage must be 0. With $s_1 s_0 = 00$, each stage as it stands generates the function $F_i = A_i$. To change the output to an OR operation, we must change the input to each full-adder circuit from $A_i$ to $A_i + B_i$. This can be accomplished by ORing $B_i$ and $A_i$ when $s_2 s_1 s_0 = 100$.

The other selection variables that give an undesirable output occur when $s_2 s_1 s_0 = 110$. The unit as it stands generates an output $F_i = A_i \odot B_i$, but we want to generate the AND operation $F_i = A_i B_i$. Let us investigate the possibility of ORing each input $A_i$ with some Boolean function $K_i$. The function so obtained is then used for $X_i$ when $s_2 s_1 s_0 = 110$:

$$F_i = X_i \oplus Y_i = (A_i + K_i) \oplus B_i' = A_i B_i + K_i B_i + A_i' K_i' B_i'$$

Careful inspection of the result reveals that if the variable $K_i = B_i'$, we obtain an output:

$$F_i = A_i B_i + B_i' B_i + A_i B_i B_i' = A_i B_i$$

Two terms are equal to 0 because $B_i B_i' = 0$. The result obtained is the AND operation as required. The conclusion is that, if $A_i$ is ORed with $B_i'$ when $s_2 s_1 s_0 = 110$, the output will generate the AND operation.

Available on notes.arpanadhikari7.com.np

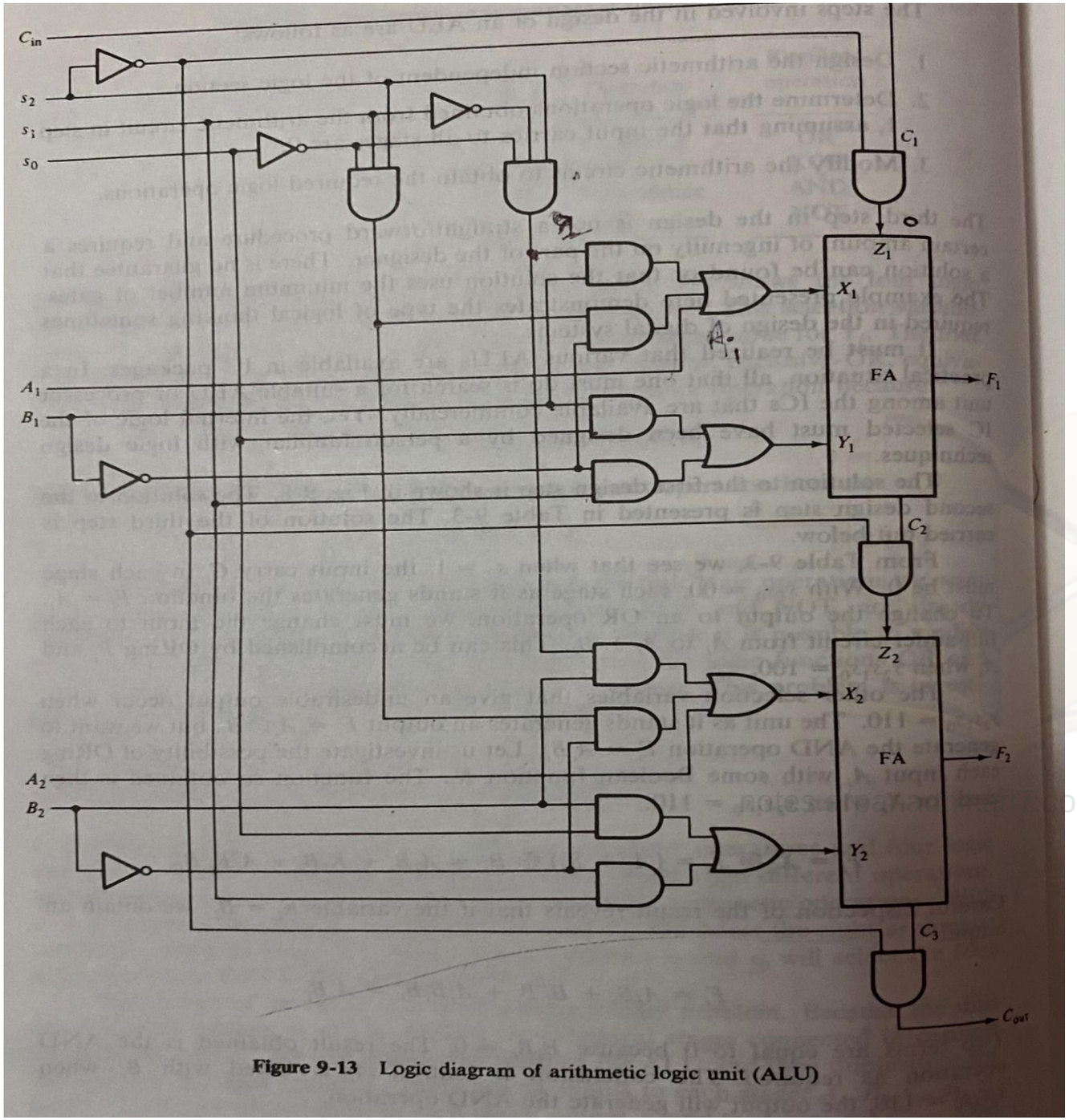



**Figure 9-13** Logic diagram of arithmetic logic unit (ALU)

Available on notes.arpanadhikari7.com.np

The final ALU is shown in Fig. 9-13. Only the first two stages are drawn, but the diagram can be easily extended to more stages. The inputs to each full-adder circuit are specified by the Boolean functions:

$$X_i = A_i + s_2 s_1' s_0' B_i + s_2 s_1 s_0' B_i'$$
$$Y_i = s_0 B_i + s_1 B_i'$$
$$Z_i = s_2' C_i$$

When $s_2 = 0$, the three functions reduce to:

$$X_i = A_i$$
$$Y_i = s_0 B_i + s_1 B_i'$$
$$Z_i = C_i$$

which are the functions for the arithmetic circuit of Fig. 9-8. The logic operations are generated when $s_2 = 1$. For $s_2 s_1 s_0 = 101$ or 111, the functions reduce to:

$$X_i = A_i$$
$$Y_i = s_0 B_i + s_1 B_i'$$
$$C_i = 0$$

Output $F_i$ is then equal to $X_i \oplus Y_i$ and produces the exclusive-OR and complement operations as specified in Table 9-3. When $s_2 s_1 s_0 = 100$, each $A_i$ is ORed with $B_i$ to provide the OR operation as discussed above. When $s_2 s_1 s_0 = 110$, each $A_i$ is ORed with $B_i'$ to provide the AND operation as explained previously.

The 12 operations generated in the ALU are summarized in Table 9-4. The particular function is selected through $s_2$, $s_1$, $s_0$, and $C_{in}$. The arithmetic operations

**TABLE 9-4** Function table for the ALU of Fig. 9-13

| Selection | | | | Output | Function |
|---|---|---|---|---|---|
| $s_2$ | $s_1$ | $s_0$ | $C_{in}$ | | |
| 0 | 0 | 0 | 0 | $F = A$ | Transfer $A$ |
| 0 | 0 | 0 | 1 | $F = A + 1$ | Increment $A$ |
| 0 | 0 | 1 | 0 | $F = A + B$ | Addition |
| 0 | 0 | 1 | 1 | $F = A + B + 1$ | Add with carry |
| 0 | 1 | 0 | 0 | $F = A - B - 1$ | Subtract with borrow |
| 0 | 1 | 0 | 1 | $F = A - B$ | Subtraction |
| 0 | 1 | 1 | 0 | $F = A - 1$ | Decrement $A$ |
| 0 | 1 | 1 | 1 | $F = A$ | Transfer $A$ |
| 1 | 0 | 0 | $X$ | $F = A \vee B$ | OR |
| 1 | 0 | 1 | $X$ | $F = A \oplus B$ | XOR |
| 1 | 1 | 0 | $X$ | $F = A \wedge B$ | AND |
| 1 | 1 | 1 | $X$ | $F = \bar{A}$ | Complement $A$ |

Available on notes.arpanadhikari7.com.np

A+L+shift.

ALU design,

1 bit ALU

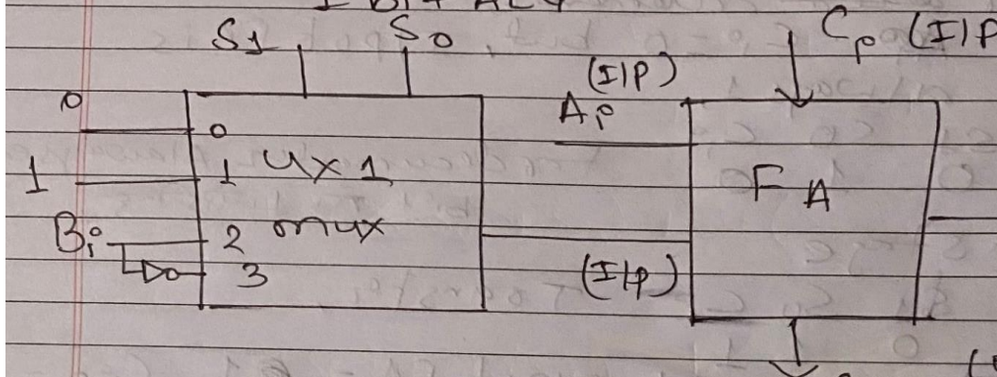



Fig:- 1 bit Arithematic Circuit.

| $S_1$ | $S_0$ | $C_{in}$ | Operations |
|---|---|---|---|
| 0 | 0 | 0 | $A_i$ (Transfer) |
| 0 | 0 | 1 | $A_i+1$ (Increment) |
| 0 | 1 | 0 | $A_i-1$ (Decrement) |
| 0 | 1 | 1 | $A_i$ (Transfer) |
| 1 | 0 | 0 | $A_i+B_i$ (Add) |
| 1 | 0 | 1 | $A_i+B_i+1$ (Add with carry) |
| 1 | 1 | 0 | $A_i+\overline{B_i}$ (Subtract with borrow) |
| 1 | 1 | 1 | $A_i+\overline{B_i}+1$ (Subtract) |

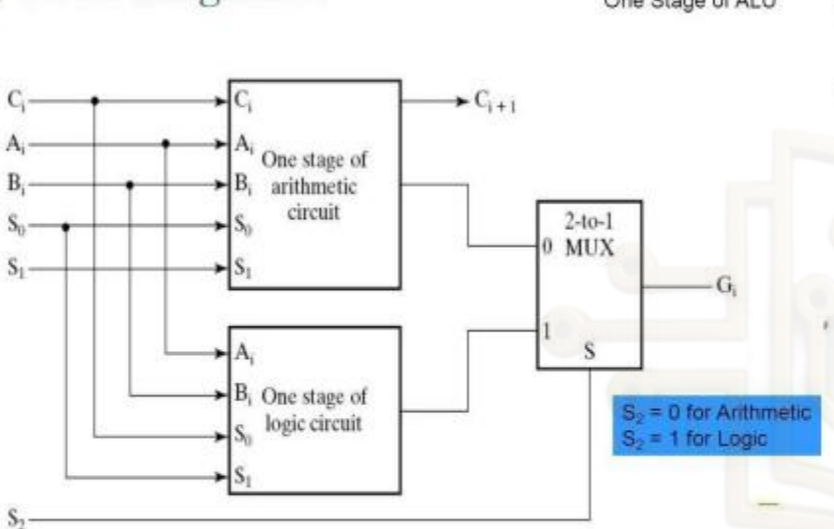
Note, $A_i+\overline{B_i}+1$ = ~~Add~~ ~~Carry~~, Hence, $A-B$

$= A+(\overline{B}+1)$ (2's complement)

Subtract with borrow.

Available on notes.arpanadhikari7.com.np

To be designed…




One stage ALU

Operation select

| $S_2$ | $S_1$ | $S_0$ | $C_{in}$ | Operation | Function |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | $F = A$ | Transfer $A$ |
| 0 | 0 | 0 | 1 | $F = A + 1$ | Increment $A$ |
| 0 | 0 | 1 | 0 | $F = A + B$ | Addition |
| 0 | 0 | 1 | 1 | $F = A + B + 1$ | Add with carry |
| 0 | 1 | 0 | 0 | $F = A + \overline{B}$ | Subtract with borrow |
| 0 | 1 | 0 | 1 | $F = A + \overline{B} + 1$ | Subtraction |
| 0 | 1 | 1 | 0 | $F = A - 1$ | Decrement $A$ |
| 0 | 1 | 1 | 1 | $F = A$ | Transfer $A$ |
| 1 | 0 | 0 | × | $F = A \wedge B$ | AND |
| 1 | 0 | 1 | × | $F = A \vee B$ | OR |
| 1 | 1 | 0 | × | $F = A \oplus B$ | XOR |
| 1 | 1 | 1 | × | $F = \overline{A}$ | Complement $A$ |

Truth Table of ALU

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Design principle of Modern system

- Efficient memory utilization
- Pipelining
- Parallel Processing
- Uses of caches at different levels
- Multicore System

notes.arpanadhikari7.com.np